Blocklist-Forecast: Proactive Domain Blocklisting by Identifying Malicious Hosting Infrastructure

Udesh Kumarasinghe
ukumaras@purdue.edu
Purdue University
West Lafayette, Indiana, USA

Mohamed Nabeel
mmohamednabe@paloaltonetworks.com
Palo Alto Networks, Inc
Santa Clara, California, USA

Charitha Elvitigala
charitha@c2si.org
C2SI
Colombo, Sri Lanka

ABSTRACT

Domain blocklists play an important role in blocking malicious domains reaching users. However, existing blocklists are reactive in nature and slow to react to attacks, by which time the damage is already caused. This is mainly due to the fact that existing blocklists and reputation systems rely on either website content or user interactions with the websites in order to ascertain if a website is malicious. In this work, we explore the possibility of predicting malicious domains proactively, given a seed list of malicious domains from such reactive blocklists. We observe that malicious domains often share the infrastructure utilized for previous attacks, reuse or rotate resources. Leveraging this observation, we selectively crawl passive DNS data to identify domains in the "neighborhood" of seed malicious domains extracted from reactive blocklists. Due to the increased utilization of cloud hosting, not all such domains in the neighborhood are malicious. Further vetting is required to identify unseen malicious domains. Along with the proximity, we identify that hosting and lexical features help distinguish malicious domains from benign ones. We model the infrastructure as a heterogeneous network graph and design a graph neural network to detect malicious domains. Our approach is blocklist-agnostic in that it can work with any blocklist and detect new malicious domains. We demonstrate our approach utilizing 7 month longitudinal data from three popular blocklists, PhishTank, OpenPhish, and VirusTotal. Our experimental results show that, our approach for VirusTotal feed detects 4.7 unseen malicious domains for every seed malicious domain at a very low FPR of 0.059. Further, we observe the concerning trend that 47% of predicted malicious domains that are later flagged in VirusTotal are identified only after more than 3 weeks to months since our model detects them.

CCS CONCEPTS

• **Security and privacy** → **Malware and its mitigation**; **Phishing**; • **Networks** → *Web protocol security*.

KEYWORDS

malicious domains, graph learning, passive DNS, domain association

This work is licensed under a Creative Commons Attribution International 4.0 License.

RAID 2024, September 30–October 02, 2024, Padua, Italy
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0959-3/24/09
https://doi.org/10.1145/3678890.3678925

ACM Reference Format:
Udesh Kumarasinghe, Mohamed Nabeel, and Charitha Elvitigala. 2024. Blocklist-Forecast: Proactive Domain Blocklisting by Identifying Malicious Hosting Infrastructure. In *The 27th International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2024), September 30–October 02, 2024, Padua, Italy*. ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3678890.3678925

1 INTRODUCTION

One of the most widely used techniques to protect Internet users from cyber attacks is to use blocklists that enumerate known malicious IPs and domains periodically. These blocklists are integrated into web browsers (e.g., Google Safe Browsing, PhishTank, OpenPhish), perimeter security appliances such as firewalls, or other application security appliances such as mail servers. When a user accesses a website, if the domain is present on the blocklist, these systems either block access or warn, indicating the harmful content. Similarly, email services block or place emails in a special folder if the email originates from a known IP address or domain that sends spam (i.e., a block-listed IP/domain).

In this work, we focus on domain blocklists. These blocklists provide efficient lookups, but the major drawback of such lists is that they are reactive [10]. More concerningly, these blocklists list malicious domains only after either content is available or a sufficient number of users access or report as a malicious website. Therefore, a window of vulnerability exists between the time of domain exhibiting malicious behavior and the time users access it. To add insult to injury, most of the malicious campaigns may last from a few hours to days. Thus, the damage is already done by the time these malicious websites are blocklisted. In contrary, a proactive approach identifies as many malicious websites as possible *before* these websites either become actively malicious or accessed by users. Further, attackers increasingly use techniques such as cloaking and CAPTCHA, presenting completely different web content, often benign, to security scanners [4]. This makes scanner-based malicious website detection ineffective at detecting highly evasive or stockpiled domains, where attackers create multiple domains and use them strategically over time. An alternative approach is required to detect these evasive malicious domains. Our objective in this work is to detect such malicious domains proactively. Figure 1 indicates the deficiency of existing approaches compared to the proposed approach.

With the increased attack automation, Internet miscreants either reuse resources or use resources from similar Internet infrastructures to operate their malicious campaigns [35]. Such behavior creates intrinsic associations among known malicious domains from

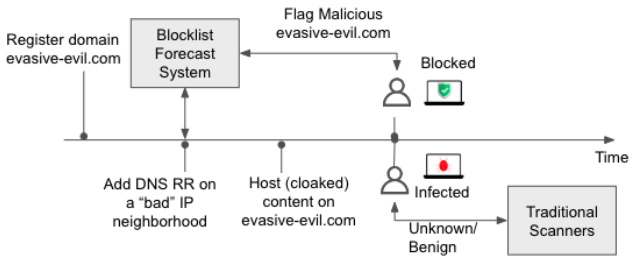


Figure 1: Our approach vs. traditional scanners

reactive blocklists and yet-to-be-discovered malicious domains. Figure 2 indicates two contrasting hosting neighborhoods for predominantly malicious and benign domains, respectively. We observe much stronger associations among malicious to malicious and benign to benign domains compared to malicious to benign domains.

One may leverage these associations to predict malicious domains proactively in the *neighborhood* of known malicious seeds. In this study, we define the neighborhood as a group of DNS entities (i.e., domains, IPs, and ASNs) relating to another entity through DNS resolutions (e.g., a neighborhood of an IP can be the multiple domains pointing to that IP). A major benefit of proactive blocklisting is that it reduces the window of vulnerability. We identify two properties that a practical and a proactive blocklisting approach should satisfy. First, it should identify malicious domains closer to their hosting time (i.e., when attackers add a DNS resource record to their zone file). Second, it should have a low false positive rate to minimize the collateral damage of incorrectly blocking benign domains.

The modus operandi of attackers is two-fold: compromise a benign domain or create their own domains to launch attacks [33]. In this work, our objective is to catch the latter before they reach users as these highly disposable domains may cause severe damage in a short time period. Thus, we use known attacker registered domains as our seed malicious domains. However, attacker registered domains are not readily available from URL blocklists. Following the findings in [33], we first use a conservative heuristic based approach to identify attacker registered domains from URL blocklists.

We then identify the attack neighborhood around each attacker registered domain. Attacker registered domains are often sinkholed, parked or taken down after an attack. Therefore, it is important to identify the attack neighborhood at the time of the attack to characterize the Internet infrastructure surrounding attack domains. To achieve this objective, we use both active and passive DNS data to discover the domains and IPs in the neighborhood. Due to the increased utilization of shared hosting services, not all domains in the neighborhood of a malicious seed domain are malicious. Hence, one needs an approach to distinguish malicious neighbors from benign ones. We observe that malicious domains in the neighborhood have discriminating infrastructure features, such as lexical formation and hosting patterns, compared to benign domains.

Following the prior research, one approach to distinguishing malicious neighboring domains from benign ones is to train a traditional supervised learner, such as Random Forest or XGBoost with the features extracted for these domains [44]. Another approach would be to use a shallow embedding technique such as Node2Vec [14] or Metapath2Vec [7] that captures the neighborhood network structure and uses a supervised downstream classifier. We find these approaches to be sub-optimal. While the first approach fails to capture the neighborhood structure, the second approach cannot use the attribute features of domains. One solution is to combine the attribute features of domains with the shallow embeddings and then train a supervised classifier. While it outperforms each independent approach, it still cannot capture non-linear relationships among domains. An approach that considers both domain features and graph structure simultaneously is likely to yield better results. In fact, graph neural networks (GNNs) are a promising model for achieving this objective. However, as shown in the literature, GNNs perform well only if the underlying graph exhibits the homophily property at least to a certain degree and node (i.e., domain and other network resources) features are discriminative. Further, careful designing of the graph is necessary for GNNs to not over-fit to a dataset and to generalize well to unseen data [41]. We address these challenges by implementing an approach that can predict malicious domains even after weeks with a very low false positive rate.

We build upon the prior research on utilizing GNNs to predict malicious domains [35, 37, 43], addressing the drawbacks of these prior approaches. Specifically, we identify the following drawbacks. First, they perform poorly with real-world domain-IP resolution data, as they do not distinguish between compromised domains and attacker registered domains [33], which weakens the homophily assumption. Second, attackers increasingly use shared hosting infrastructure to host their malicious domains due to various reasons and to reduce the maliciousness of the neighborhood. Prior approaches fail to address this trend, resulting in a low detection rate. 3. They do not generalize to unseen data as they often use a static dataset disregarding the temporal properties. To address these drawbacks, our primary goal is to build a practical system that is deployable to detect malicious domains in real time when they resolve to valid IP addresses. In summary, our contributions are as follows;

- We build a block-list agnostic pipeline to extract seed malicious domains from known block-lists. We predict 4.7x more malicious domains in the neighborhoods of seed malicious domains daily at a very low false positive rate (FPR) of 0.001. Our model achieves, on average, a high accuracy of 94.3%, and a high F1 score of 0.943.
- We analyze and evaluate the classifier in terms of various classification parameters, including training set size, the robustness of the model against random and adversarial noise, and the retrain window.
- With the trained models, we analyze seven months of longitudinal malicious URLs from VirusTotal, PhishTank, and OpenPhish. We demonstrate our models are generalizable across blocklists and over time.
- We analyze the proactiveness of our models and show that our models predict 37% of malicious domains days and months

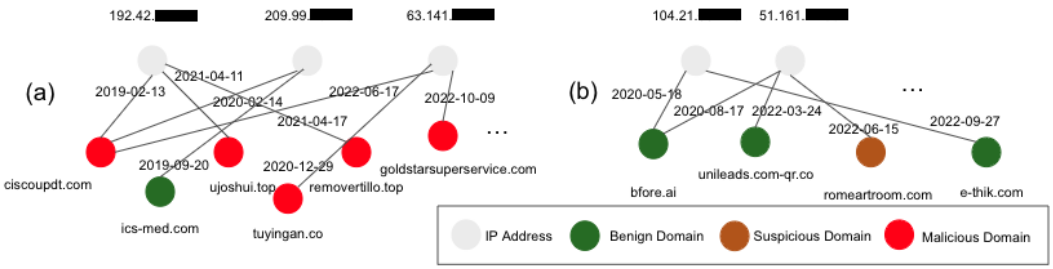


Figure 2: Two Different Hosting Patterns: (a) Hosting infrastructure around a malicious neighborhood, (b) Hosting infrastructure around a benign neighborhood. These figures show that the IPs (redacted) that hosted malicious domains are likely to host more malicious domains, while IPs that hosted benign domains are likely to host benign domains over time.

ahead of popular URL blocklists. Concerningly, these blocklists have a blind spot for many malicious domains (VirusTotal does not flag 49% of the detected malicious domains).

The rest of the paper is organized as follows. Section 2 provides preliminary information on DNS and network graphs, and graph embedding methods. In Section 3, we provide an overview of our proposed detection system. Our data collection methodology and experimental setup are described in Section 4 and Section 5. Section 6 presents the empirical results of the proposed methods and their comparison against the state-of-the-art. Sections 7, 8 and 9 analyze different aspects of the study such as real-time predictions, predictions over a period of time and properties of the classifier. Section 11 critically compares the work related to ours. Finally, we conclude the paper by mentioning the contributions, limitations, and future work.

2 BACKGROUND

In this section, we introduce DNS data and network graph, the data feeds we use and provide a high-level primer on popular graph embedding methods to make the paper self-sufficient.

2.1 DNS Data and Network Graph

Different artifacts of the Domain Name Service (DNS) and their interconnections can be represented as a network graph. The nodes of the graph define the artifacts, such as domains, IPs, and ASNs, while edges identify relationships among them. Relationships connecting the DNS artifacts can be of different types, such as the domain resolving to an IP, the domain being a sub-domain of another, or the domain being similar to another. Thus, a heterogeneous information network (HIN) can represent the underlying data structure of the network graph, which is formally defined as follows:

Definition 2.1. A **Heterogeneous Information Network** [32] is defined as a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{A}, \mathcal{R})$, where $\mathcal{V}$ and $\mathcal{E}$ denote the set of nodes and edges, respectively. $\mathcal{G}$ is associated with a node type mapping $\phi : V \rightarrow A$ and an edge type mapping $\psi : E \rightarrow R$ where the number of node types $|\mathcal{A}| > 1$ or the number of edge types $|\mathcal{R}| > 1$.

2.2 Domains and Data Feeds

2.2.1 Private and Public Apex Domains. We categorize domains into private and public domains based on their apex (i.e., e2LD - effective Second Level Domain). An apex domain is public if the apex domain owner does not create and control the sub-domains or path suffixes. For example, 000webhostapp.com, github.io, weebly.com and medium.com are public apex domains. Similarly, an apex domain is private if the apex owner creates and manges the sub-domains. For example, nsa.gov, ai.google and stanford.edu are private apex domains.

2.2.2 Passive DNS. Passive DNS (PDNS) [40] captures traffic by sensors cooperatively deployed in various DNS hierarchy locations. In our research, we use Farsight PDNS DB [8] to extract Domain-IP resolutions and hosting features. Farsight PDNS DB captures and stores all domain resolutions that it captures passively from all around the world, providing a representative population of domain resolutions. We note that we collect only publicly available aggregated network resolutions which do not contain personally identifiable information.

2.2.3 VirusTotal. VirusTotal (VT) is one of the most popular URL scanning services widely used in both academia and industry. VT aggregates the verdict from 90+ URL scanners to provide a report for each URL scanned. VT feed aggregates all the URLs scanned from everywhere in the world. Similar to prior research [6] and industry practice, we use attacker created domains from this feed as seed malicious domains.

2.2.4 PhishTank. PhishTank (PT) [3] is an open repository of phishing URLs where users submit URLs to check if they are phishing or not. Then, if a URL receives enough votes from the community, it becomes "verified". PT compiles and publishes two lists: (1) URLs submitted by its users and (2) verified URLs. We collect the latter every 1 hour and build a timestamped URL blocklist.

2.2.5 OpenPhish. OpenPhish (OP) [2] is a proprietary phishing repository which updates its phishing URL list every 5 minutes. We collect OpenPhish URLs every 5 minutes and build a timestamped URL blocklist.

2.2.6 Alexa Top 1 million. The Alexa [1] top list publishes a daily list that reflects the popularity of domains [1].

2.3 Graph Embedding Methods

Graph embedding is a representational learning mechanism that generates features for either individual nodes, edges, or entire graphs. These methods learn representations by incorporating neighborhood structure, node features, or both. Such representations are invaluable for their use in downstream machine-learning tasks. Embedding methods assume that the graph under consideration exhibits the homophily property, where nodes closer to one another are similar, but nodes far away from one another are dissimilar. There are two types of embedding methods: shallow embedding and deep embedding.

2.3.1 Shallow Embedding Methods. Node2Vec [14] is a shallow embedding method that maps individual graph nodes into high-dimensional vectors. It performs biased short random walks to generate sequences which will optimize the node embedding given the intuition that nodes have similar embedding if they co-occur in such sequences. Random walks can be manipulated using two hyperparameters p and q. p is the likelihood of revisiting the immediately previous node while q is the likelihood of visiting the adjacent neighbors. Node pairs from the generated random walk sequences are fed into a skip-gram model followed by a Soft-max layer. Embedding is extracted by concatenating model weights of the hidden layer corresponding to each node in the graph.

Metapath2Vec [7] is also a shallow embedding method but is more applicable in a heterogeneous graph. Instead of biased random walks, it uses predefined meta-paths for the random walks. Hence, it can make random walks on a heterogeneous graph that adhere to the semantic behaviors of the graph. The generated sentences from the random walks are then fed into a skip-gram model similar to Node2Vec in order to generate a high dimensional vector representation of each node in the graph.

Even though shallow embeddings demonstrated their capability to generate representative embeddings, they come with a few caveats. First, they do not share parameters between nodes during the embedding process, which limits these methods only to a transductive setting. Hence, the embedding model has to be re-trained every time the graph evolves. Second, shallow embeddings do not incorporate node features, discarding vital information.

2.3.2 Deep Embedding Methods. Graph Neural Networks (GNN) such as GCN [19] and GraphSAGE [15] leverage both node features together with network structure to learn representations of the graph. GNNs achieve this by defining a propagation rule that aggregates the feature of the node neighborhood along the graph structure. In a node classification task, GCN learns the graph structure in a semi-supervised setting, but one can extend it to generate embeddings by contrastive learning. Thus, we use DeepGraphInfomax (DGI) [38], a contrastive learning approach for GCN. DGI generates the negative sample by shuffling the node features existing graph structure. Additionally, DGI incorporates a summarizing function that summarizes the global structure of the graph.

[1]Alexa Top 1 million API was discontinued at the end of 2022. One may use an alternative popularity list, such as Tranco, which is also updated daily.

However, GCN requires its propagation rule to be applied on the entire graph as a single batch, which is infeasible given the large scale of the DNS graph. GraphSAGE [15] extends GCN into a scalable solution by proposing a neighborhood sampling technique. Thus, it enables node representation learning in a batch learning approach. Furthermore, the computation graph generated by neighborhood sampling provides a better fit for inductive predictions.

The propagation rule for GraphSAGE is defined as follows;

$$\mathbf{h}_i^{(k+1)} = \sigma\left(\mathcal{W}_0^{(k)}\mathbf{h}_i^{(k)} + \mathcal{W}_1^{(k)} \cdot mean_{j\in\mathcal{N}i}\mathbf{h}_j^{(k)}\right) \quad (1)$$

where $\mathcal{N}(i)$ is the neighborhood of node i, $\mathcal{W}_0$ is a trainable weight for the self node while $\mathcal{W}_1$ is a trainable weight for the neighborhood. $\sigma(\cdot)$ is a element-wise activation function such as the $ReLU$.

GCN and GraphSAGE are designed for homogeneous graphs. Hence, they fail in our case as our knowledge graph is heterogeneous to depict different entities in the DNS. Relational GCN (RGCN) [29] is a variation that supports learning on graphs with multiple relationship types. RGCN tackles the heterogeneity by defining separate learnable weights for each edge type. Thus, it enables to apply convolution operations on the graph without losing information available in different relationships in the heterogeneous DNS graph.

Propagation rule of 1 is extended to the following:

$$\mathbf{h}_i^{(k+1)} = \sigma\left(\sum_{r\in\mathcal{R}}\sum_{j\in\mathcal{N}^r(i)} \frac{1}{|\mathcal{N}^r(i)|}\left(\mathcal{W}_r^{(k)}\mathbf{h}_j^{(k)}\right) + \mathcal{W}_0^{(k)}\mathbf{h}_i^{(k)}\right) \quad (2)$$

where $\mathcal{N}^r(i)$ is the neighborhood of node i under edge type $r \in \mathcal{R}$.

3 OVERVIEW

Figure 3 shows the overall workflow of predicting new malicious domains from malicious seed domains extracted from the three blocklists considered in this work, namely VirusTotal, PhishTank and Openphish.

All three blocklists utilized in this work are URL blocklists. These malicious URLs are either hosted on compromised domains or attacker registered domains. Since our goal is to predict new attack domains, our seed malicious domains must be attacker registered domains. However, attacker registered domains are not readily distinguishable from these URL blocklists. We utilize a conservative heuristic based approach to extract seed attacker registered domains from these malicious URLs. We manually verify random samples of these seed domains and ascertain they are indeed attacker registered domains [2].

Figure 3 shows the methodology we utilize to identify seed attacker registered domains from blocklisted URLs. We first identify a likely set of attacker registered domains for each blocklist. For VirusTotal, we select those URLs having at least 5 positives following prior research [31, 39]. For Phishtank and OpenPhish, we select only those URLs that are manually verified. We extract the FQDNs for these URLs and exclude those hosted on public hosting services such as wordpress.com, godaddysites.com, and 000webhostapp.com

[2]To flag attack domains manually, we utilize a rubric that assesses the historical records associated with the website and current status.

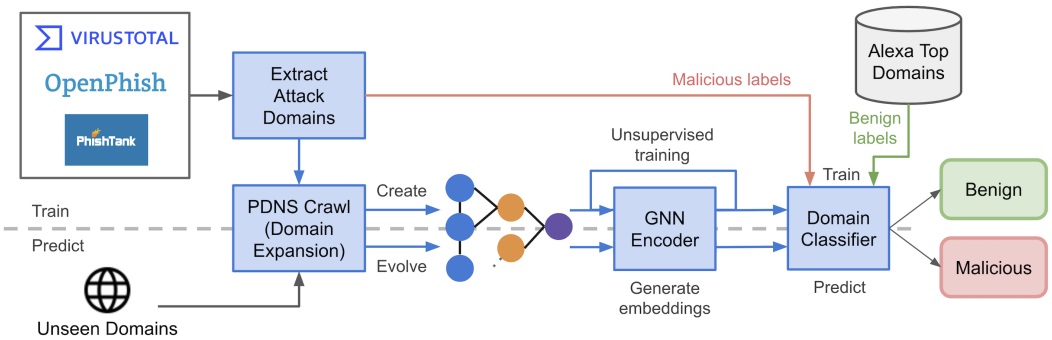


Figure 3: Overall Workflow of Predicting Malicious Domains

as they are hosted on reputed infrastructures controlled by the public hosting service providers [3]. The rationale for the removal is public domains they are less likely to be associated with the infrastructure on which attacker registered domains are hosted.

Next, we identify compromised apex domains from the remaining and filter them out. We observe that compromised domains are frequently hosted in neighborhoods that have many other benign domains. Having compromised apexes as malicious seeds increase the likelihood of incorrectly classifying benign domains in these neighborhoods as malicious. Compromised domains are likely to either live long or persist in Alexa top 1 million domain list for relatively a long time period. Following these observations, we describe how to identify compromised domains in Section 4. At the end of this pipeline, we identify a set of attacker registered domains from each blocklist.

We then expand the seed attack domains by selectively crawling the PDNS repository to identify domains in the neighborhood of the seed malicious domains. We describe the algorithm we utilize for crawling in Section 4.2.

For each blocklist, in Section 4.2, we construct a HIN consisting of domains, IPs, and additional infrastructure nodes. As described in Section 5, we then learn an encoder that generates an embedding for each domain that captures its neighborhood (both the structure and the features).

For the HIN constructed, we identify a set of benign domains by checking against pre-processed Alexa top 1 million domains as described in Section 4.1.2.

Given an updated or new HIN constructed from the same blocklist, we utilize the learned encoder to generate embeddings for these domains. Along with the seed attacker registered domains and identified benign domains as ground truth, we utilize these embeddings to train a downstream supervised learner.

4 DATA COLLECTION AND GRAPH CONSTRUCTION

4.1 Ground Truth Collection

4.1.1 Malicious Ground Truth. We use three popular blocklists/reputation systems to extract attack domains: VT, PT, and OP. In practice and research, the VirusTotal URL feed is utilized as a blocklist with an appropriate threshold indicating the number of scanners marking a given URL as malicious. As mentioned previously, we use one of the most conservative threshold of 5 scanners [31, 39] to select malicious URLs from VT. We retrospectively collect daily malicious domains for each of these blocklists from Oct 2020 to Apr 2021.

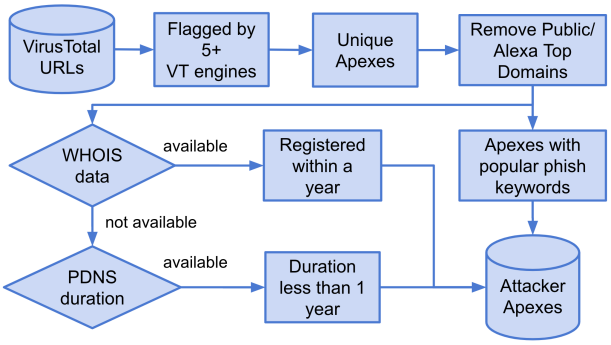


Figure 4: Attacker registered domain filtering pipeline

We execute the pipeline illustrated in Figure 4 to extract attacker registered domains from the daily identified malicious domains. First, we collect URLs that are seen for the first time and are flagged as malicious by at least five VT scanners. After extracting unique malicious apexes, any apexes present in the public domain lists or Alexa top domains are removed as they are more likely to be compromised domains or less likely to be attacker registered apex domains. Next, any apex that embeds a popular phishing keyword [9, 20] is directly added to the attack domain list. Finally, the remaining malicious apexes are further refined by two temporal constraints based on the observation that attack domains are short lived [26, 33]: domain WHOIS registration or passive DNS duration is within a year. Specifically, we apply the following heuristics to identify likely compromised apexes. First, domains that consistently persist in Alexa top 1 million domains list for more than 1 month are excluded. Then, the domains with a registration period of more than 2 years are excluded. If the domain's WHOIS registration record is

[3]We utilize a list of 93K+ public domains containing web hosting, document sharing sites, social media sites, and proxy services made avaiable by prior research [33]

inaccessible, we utilize the duration of the footprint in the PDNS repository as an estimate [4]. We take random samples of these attack domains and manually verify that they are in fact attacker registered domains. We follow a similar pipeline for PT and OP, except that all malicious URLs are considered to extract the unique apexes. Table 1 shows the average number of attacker registered domains we extract from each blocklist on a daily basis.

4.1.2 Benign Ground Truth. Following previous research findings [28], we utilize the Alexa top 1 million 1 month consistent list as our benign ground truth. Hence, instead of weakly assuming that Alexa Top 1m is benign, we utilize the observation that malicious domains are short-lived, thus minimizing the chance of appearing in the top list for an extended period. Note that the benign domain list is not utilized to expand the graph as our goal is to find high toxicity neighborhoods.

4.2 Passive DNS Expansion

One possible approach to building a DNS resolution graph is to consider all domain-IP resolutions around the world. While this covers all the domains, this approach suffers from two drawbacks: 1. The scale of the problem demands a huge amount of computing resources, and 2. The return on the investment is too low as the vast majority of domains are benign. In other words, the toxicity (maliciousness) of the graph is too low to be practical. A better approach is to use the attacker registered domains as the seed and identify the hosting infrastructure (i.e., IP addresses and domains) around these seed domains since, as mentioned earlier, malicious domains over time tend to utilize similar network resources. We observe that attackers increasingly utilize shared hosting to launch their attacks. Since these shared IPs host many unrelated benign domains which are often long-lived, we make a selective crawling of domains to only extract those domains that are newly hosted on a given IP address. Specifically, we follow the below steps to build the expanded DNS resolution graph:

(1) Collect the IPs hosting the seed malicious domains.
(2) Collect the most recent 100 [5] domains hosted on each IP collected in step 1.
(3) Collect the recent IPs hosting the domains collected in step 2.

Table 1 shows the number of DNS resolutions, domains, and IP addresses we crawl on average based on the seed malicious domains on a daily basis.

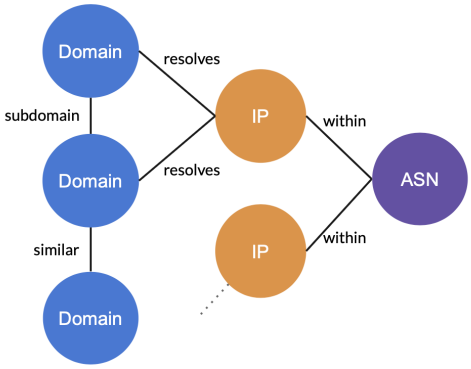
4.3 HIN Construction

We use the collected domain-IP resolutions to build a heterogeneous knowledge graph consisting of node types domains, IPs, and ASNs (Autonomous System Numbers). We identify the following 4 types of relationships among these nodes (Figure 5a):

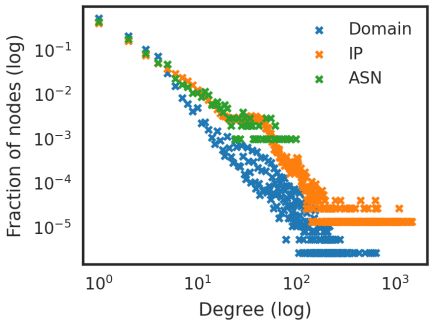
[4]We selected the thresholds for the heuristics after manually analyzing 100 attack domains. We remark that the heuristics based approach is practical but conservative. One may gather additional attack domains by training a machine learning model [33].
[5]We empirically select 100 based on the maliciousness of the resulting graph. Depending on the computational resources available, one may select a higher threshold.

Table 1: Statistics of different block-list data and expanded domains, IPs and resolutions (monthly average)

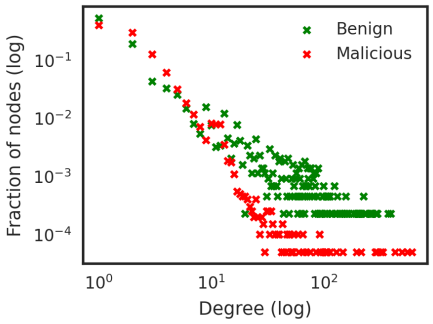
Block-list	Attack Dom.	Domains	IPs	Resolutions
VirusTotal	40,888	740,823	212,273	2,996,028
OpenPhish	11,357	166,890	58,112	439,554
PhishTank	6,260	117,421	47,752	389,213



(a) HIN containing domain, IP, and ASN nodes



(b) DNS



(c) Benign vs. Malicious Domains

Figure 5: HIN schema and degree distribution of the two datasets

4.3.1 domain-resolve-IP. identifies the relationship between a domain and IP if the domain is hosted on the IP. This relationship is readily available from the DNS resolutions.

4.3.2 domain-similar-domain. captures the character level similarity between two domains. In order to find the domain similarity, we utilize the tri-gram model to process domain names and embed each domain in a high level representation using TF-IDF vectorization. Using this representation, we use cosine similarity to identify domains with character level similarity.

4.3.3 domain-subdomain-domain. encodes the relationship between a domain and another domain if the domain is a sub-domain of the apex. If the apex is not present, we extract the apex and create this relationship to explicitly encode this relationship in the graph.

4.3.4 IP-within-ASN. relationship between an IP and an ASN is created if the IP belongs to a registered ASN. ASNs are extracted from the Maxmind Database for the IPs available in the domain-IP resolutions.

Figure 5 (b) and 5 (c) show the node degree distributions for our DNS knowledge graph. We make several observations from these statistics: (1) The degrees follow the power law distribution similar to many other real-world graphs, (2) The degrees of IP nodes are higher than that of domains likely due to the proliferation of shared hosting.

5 EXPERIMENTAL SETUP

In this section, we define the experiments to validate the performance of the proposed model by first comparing different DNS graphs, comparing varying embedding mechanisms, and detailing the experimental setup used.

5.1 DNS graphs

To identify the optimal graph structure that is capable of capturing domain behavior, we carry out the malicious domain detection task on three different DNS graph schemes as follows:

5.1.1 Domain Graph. A homogeneous DNS graph that contains only domain nodes and relationships between them, specifically *domain-subdomain-domain* and *domain-similar-domain* relationships.

5.1.2 Domain-IP Graph. A heterogeneous DNS graph that contains both domain nodes and IP nodes. In addition to the relationships in the Domain graph, this schema includes undirected *domain-resolves-IP* relationships.

5.1.3 Domain-IP-ASN Graph. A heterogeneous DNS graph that contains domain nodes, IP nodes, and ASN nodes. This schema includes undirected *IP-within-ASN* relationships in addition to the relationship types in the Domain-IP graph.

In each graph, we discard insignificant nodes according to the graph pruning rules mentioned in [21].

5.2 Embedding methods

We experiment with multiple graph node embedding methods to identify the optimal approach in terms of classification performance.

5.2.1 No embedding. No graph node embedding method is applied and domain features are used directly for the classification.

5.2.2 Shallow embedding methods. Transductive shallow embedding methods such as Node2Vec and Metapath2Vec are applied to generate a representation for the domain nodes.

5.2.3 Deep embedding methods. Inductive GNN based unsupervised models that incorporate DeepGraphInfomax (DGI) such as GraphSAGE and HinSAGE are applied to generate a representation for the domain nodes.

5.3 Setup

We implement the proposed system in Python 3 [6]. The domain embedding models for the DNS graph are implemented based on PyTorch Geometric [12], a framework for learning on graph structures. Domain embedding classifier is implemented as a Random Forest classification based on the implementation of sklearn [27] machine learning library. Random Forest is selected based on its superior classification performance after spot checking other popular algorithms XGBoost, Naive Bayes, K-Nearest Neighbors, support vector machine, logistic regression, multi-layer perceptrons, and classification trees. Table 2 lists the metrics used in the experiments.

Table 2: Performance metrics for evaluation

Metric	Description
TP	# of domains correctly classified as malicious
TN	# of domains correctly classified as benign
FP	# of domains mistakenly classified as malicious
FN	# of domains mistakenly classified as benign
$Precision$	$TP/(TP+FP)$
$Recall$	$TP/(TP+FN)$
FPR	$FP/FP+TN$
ACC	$(TP+TN)/(TP+TN+FP+FN)$
$F1$	$2 \times Precision \times Recall/(Precision+Recall)$
AUC	Area Under the ROC Curve

6 RESULTS

In this section, we present the empirical results of the experiments carried out. First, we compare the performance of different DNS graphs and embedding methods. Then, we compare the proposed model against the state-of-the-art malicious domain detection approaches. Further, we evaluate the generalization of the proposed model on temporally different datasets and on different blocklists. Finally, we investigate the proposed model's sensitivity to hyperparameters.

Table 3: Performance of embedding methods on different DNS graphs

Entities	Embedding	Metrics					
		Accuracy	F1	Precision	Recall	FPR	AUC
Domain	Raw Features	0.795	0.792	0.816	0.769	0.177	0.872
	Label Propagation	0.749	0.660	1.000	0.494	0.000	-
	Node2Vec	0.838	0.832	0.859	0.808	0.132	0.931
	GraphSAGE	0.872	0.872	0.872	0.872	0.128	0.940
Domain-IP	Node2Vec	0.914	0.919	0.870	0.973	0.144	0.965
	Metapath2Vec	0.903	0.908	0.855	0.969	0.162	0.938
	GraphSAGE	0.912	0.913	0.902	0.925	0.101	0.972
	HinSAGE	0.939	0.939	0.933	0.945	0.067	0.983
Domain-IP-ASN	Node2Vec	0.920	0.924	0.893	0.956	0.115	0.970
	Metapath2Vec	0.902	0.909	0.852	**0.974**	0.170	0.936
	GraphSAGE	0.933	0.935	0.917	0.954	0.087	0.979
	HinSAGE	**0.943**	**0.943**	**0.942**	0.944	**0.059**	**0.983**

6.1 Embedding Performance

Table 3 presents the results of the three different embeddings on the three different graphs. We observe a noticeable improvement in graph embeddings over raw domain features and label propagation. A key reason is that these approaches do not take full advantage of the graph structure. From all the embedding mechanisms, the performance of the classifier improves significantly when the IP nodes and their relationships are added to the domain graph. The classifier performance is further improved when the ASN nodes and their relationships are included in the Domain-IP graph. A key reason is that the micro-level relationships (D-IP-D) and macro-level relationships (D-IP-ASN-IP-D) that help graph algorithms learn different aspects of the domain associations resulting in clear benign and malicious segmentation in the embedding space.

When comparing the different embedding methods, deep embeddings perform better compared to shallow embeddings. A key reason for this observation is that deep embedding methods consider both graph topology as well as node features whereas shallow embedding methods consider only the graph topology to generate embeddings.

Furthermore, the Table 3 shows that the heterogeneous embedding models (HinSAGE) perform better than the homogeneous models (GraphSAGE, Node2Vec) in general (best results highlighted in boldface). A heterogeneous model assigns different importance to different edge types (e.g., Domain-IP, Domain-Domain) whereas a homogeneous model treats all edges equally. In practice, we observe that these different relationships are in fact not of equal strength. For example, a Domain-Domain relationship is stronger than a Domain-IP relationship. The results support this observation.

6.2 Comparison with the state-of-the-art

In order to evaluate the performance of the proposed method, we compare it with the state-of-the-art graph-based malicious domain detection methods. Table 4 presents the performance of the graph classification models. Note that we do not compare against non-graph based malicious domain detection approaches such as Exposure [5] or Predator [16].

6.2.1 Ringer [23]. builds a homogeneous domain graph with the relationships between domain pairs if the pair shares clients or IPs. Graph Convolutional Network is trained end-to-end on the domain graph in a semi-supervised approach by utilizing a subset of labeled domains.

6.2.2 HGDom [37]. generates homogeneous domain sub-graphs from DNS HIN by using metapaths based on the observations of adversary behavior. A variant of GCN, namely MAGCN is used for malicious domain classification in a semi-supervised setting. MAGCN applies GCN to each subgraph and aggregates using an attention mechanism.

It should be noted that as implementations and datasets of these state-of-the-art works are not available publicly, we reproduce them to the best of our knowledge by following the respective works. Reimplementations of the compared works are released together with this work's artifacts for reproducibility. Furthermore, empirical results are calculated on the DNS graph we have constructed, which results in an upper bound in their performance. As the results show, our HinSAGE with DGI based model consistently outperforms the two models.

Table 4: Comparison with the state-of-the-art

Method	Metrics					
	Acc.	F1	Prec.	Recall	FPR	AUC
Ringer [23]	0.707	0.662	0.798	0.566	0.147	0.857
HGDom [37]	0.864	0.869	0.851	0.888	0.159	0.919
HinSAGE	**0.943**	**0.943**	**0.942**	**0.944**	**0.059**	**0.983**

[6]Code is available at https://github.com/ThatUdeshUK/BlocklistForecast

6.3 Generalization over time and over different blocklists

In order to evaluate the generalization of the method in different time periods, we evaluate the performance of the proposed model on monthly VirusTotal, PhishTank, and OpenPhish data collected from Oct 2020 to Apr 2021. For each month, the aforementioned data are used on a HinSAGE with DGI model to classify malicious domains of static domain-IP graphs. We use 5 fold cross validation to evaluate the model performance. Figure 6 shows the respective ROC curves of each month of VirusTotal data. Further, Table 5 presents the high and consistent F1 scores and AUC scores for each month of data. We observe that the performance on the VirusTotal data is slightly better than that on PhishTank and OpenPhish data. We attribute this difference to the graph sizes where PhishTank/OpenPhish graphs are much smaller compared to the VirusTotal graphs. These results clearly show that our approach generalizes across different time periods and different blocklists.

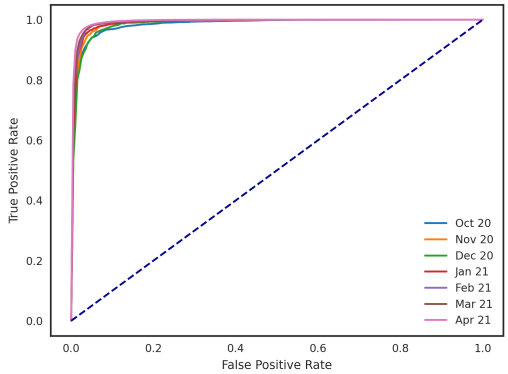


Figure 6: ROC curves of malicious domain predictions of months from Oct. 2020 to Apr. 2021.

Table 5: Performance on different blocklists in each month

Month	VirusTotal		OpenPhish		PhishTank	
	F1	AUC	F1	AUC	F1	AUC
Oct 2020	0.942	0.984	0.900	0.960	0.901	0.961
Nov 2020	0.954	0.988	0.908	0.967	0.906	0.965
Dec 2020	0.946	0.986	0.894	0.957	0.893	0.959
Jan 2021	0.959	0.991	0.902	0.962	0.893	0.956
Feb 2021	0.966	0.991	0.916	0.969	0.907	0.965
Mar 2021	0.966	0.992	0.904	0.965	0.902	0.967
Apr 2021	0.969	0.994	0.900	0.958	0.891	0.955

Furthermore, we evaluate the performance by crossing the model between different blocklists. That is, we predict malicious domains in the PhishTank dataset using the embedding model trained only using OpenPhish data from the same month. Empirical results show that PhishTank malicious domains can be predicted with a high F1 score of 0.85 (0.95 AUC) by the model trained on OpenPhish in the month of Oct. 2020. This shows that these models capture transferable knowledge on malicious domains.

7 REAL-TIME PREDICTION

Compared to the state-of-the-art, our model is able to produce node embeddings inductively (i.e., without retraining) for unseen domains. Consequently, this improves both the efficiency and the practicality of the detection model.

The inductive property of the HinSAGE model allows predictions to be made in near real-time. In contrast to transductive graph representation learning mechanisms, inductive models are able to generate representations for unseen domain nodes without retraining the model on the entire graph. Thus, the HinSAGE model can be trained once and utilized for future malicious domain predictions until its performance degrades over time. Here, we measure the concept drift of the inductive HinSAGE model in order to identify the schedule for retraining.

The model is trained using the data from the first 7 days of the month. DNS graph is generated and embeddings for the domain nodes are extracted using HinSAGE. Then, the Random Forest model is trained to build the downstream prediction model. Each test set of the size of 5 days is generated by collecting the first seen resolution after an incremental interval from the training data. Resolutions from the test data are updated on the training graph, embeddings are generated by forward passing through the trained HinSAGE model and finally predicted using the trained Random Forest model. Figure 7 shows the performance of the inductive models for each month. We observe that models consistently perform closer to the peak performance for 10+ days suggesting that one does not require to retrain the model often. We recommend re-training on a weekly basis to maintain a high classification performance over time.

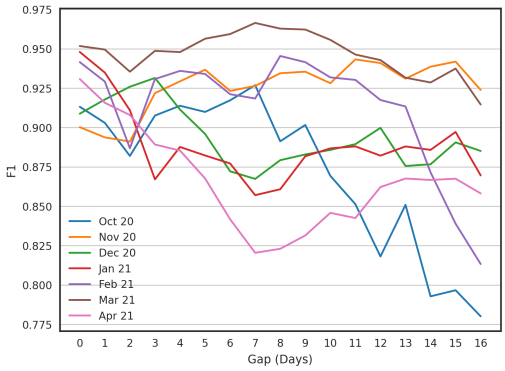


Figure 7: F1 score of Random Forest classifier for test sets after incremental intervals.

8 POST ANALYSIS

8.1 Monthly Detected Malicious Domains

Table 6 shows the number of malicious domains detected by our model on monthly basis for the three blocklists. At a very low FPR of 0.001, for every seed malicious domain in VirusTotal, OpenPhish, and Phishtank, our model detects 4.7, 0.8, and 0.7 new malicious domains not identified by these blocklists respectively. We attribute the difference in the detection percentage for different blocklists to

Table 6: Statistics of the predicted malicious domain in each month (FPR=0.001)

Month	Block-list	Malicious seeds	Malicious predictions
Oct 2020	VirusTotal	32,162	100,650
	OpenPhish	11,060	2,905
	PhishTank	6,964	2,772
Nov 2020	VirusTotal	35,228	134,031
	OpenPhish	13,140	8,516
	PhishTank	8,441	4,797
Dec 2020	VirusTotal	27,331	93,737
	OpenPhish	9,096	5,018
	PhishTank	396	1,032
Jan 2021	VirusTotal	43,141	208,498
	OpenPhish	10,894	7,518
	PhishTank	7,905	3,997
Feb 2021	VirusTotal	40,117	187,801
	OpenPhish	10,401	11,087
	PhishTank	6,872	5,554
Mar 2021	VirusTotal	53,234	277,489
	OpenPhish	14,501	21,758
	PhishTank	7,117	7,916
Apr 2021	VirusTotal	55,008	342,144
	OpenPhish	10,411	5,639
	PhishTank	6,129	2,286

the size of the resulting graphs (the number of nodes and edges) utilized to train the model.

8.2 Proactiveness in Blocklisting

Here, we analyze the ability of the proposed model to proactively predict blocklisted domains. We use the 2020 Oct. first week (1-7) data to train the model. Our model predicts 74,901 malicious domains at the high threshold of $t = 0.9$ and low $FPR = 0.005$. Then, predicted malicious domains are compared with first-seen domains that appeared in the future across VirusTotal and the two different blocklists, OpenPhish and PhishTank.

Figure 8a shows the daily detected malicious domains by VirusTotal during the rest of the month of Oct 2019. Figure 8b shows the detected malicious domains by the three blocklists in the following months from Nov 2020 to Apr 2021. We identify several interesting and concerning behaviors of these blocklists. First, VirusTotal fails to detect 49% of the predicted domains. Our manual analysis of samples of these missed predicted domains shows that many of these malicious domains are short-lived, parked, or cloaked. Table 8 shows a small sample of malicious domains that VirusTotal fails to flag as malicious at the time of the study. Further, as shown in Figure 8c, it is concerning to note that 47% of malicious domains detected by VirusTotal are reported 3 weeks after our model predicts them. Additionally, there are many domains detected by VirusTotal months after our initial predictions. In fact, a significant portion is flagged outside of our study period. Table 7 shows a small sample such domains. We observe similar concerning trends with the other two blocklists.

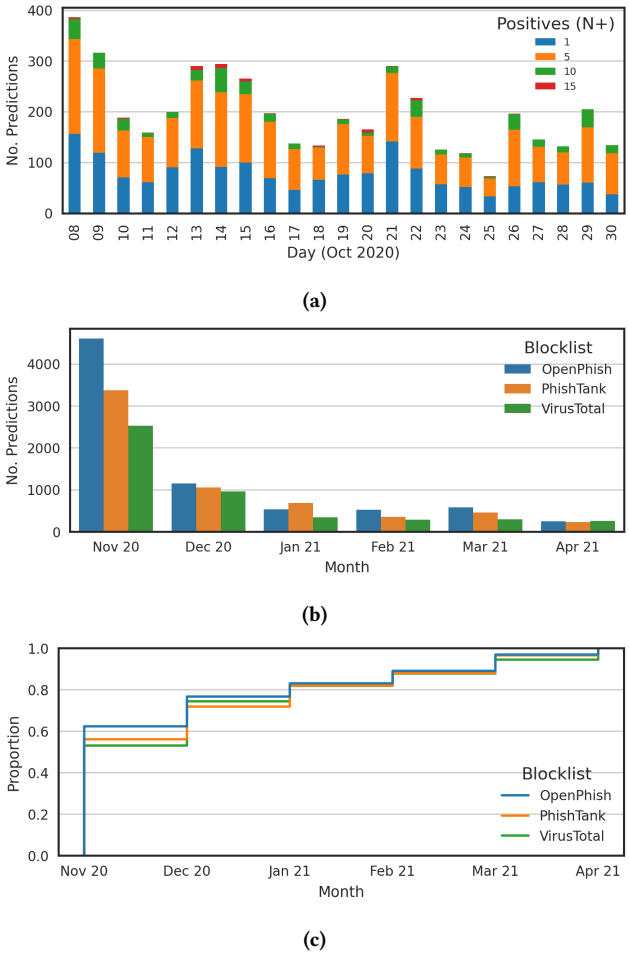


Figure 8: (a) Proactive VirusTotal predictions on different months labeled by number of positives, (b) Proactive OpenPhish and PhishTank predictions on future months, (c) Percentage of detected malicious domains over time

Table 7: Examples of VT detected domains outside of the study period

Domain	Creation Date	VT First Detected Date
greytune.com	2019-02-28	2022-05-24
gronberger.com	2020-02-10	2022-06-01
recipesonlinenow.ga	N/A	2022-06-03
cbdbyvet.com	2018-12-05	2022-06-04
fintrade.email	2018-02-17	2022-06-15

Table 8: Examples of VT not detected but malicious (manual inspection)

Domain	Duration (Days)	Status
myshipflex.com	303	NX
artistdialect.xyz	36	NX
reouj-ceval.com	N/A	NX
ceoalmomani.com	459	Deleted
stratchexplode.com	1	NX
accutacne.com	N/A	Deleted

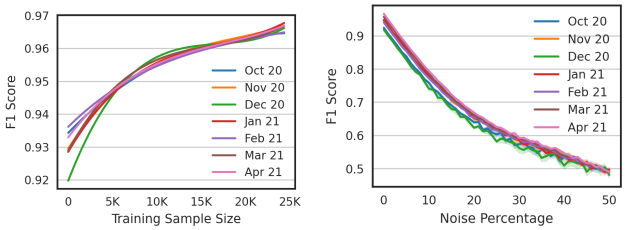
9 CLASSIFIER ANALYSIS

9.1 Size of Training Data

To address how much training data is required to train the model with desirable performance, the Random Forest classifier is trained with different training data sizes of labeled graph embeddings generated from HinSAGE. The process is carried out on data from different months. Furthermore, the experiment with each dataset size is carried out 3 times to get the average metrics. Additionally, SMOTE oversampling is used for the months that have a comparatively less number of labeled embeddings. As visualized by the Figure 9a, improvement of model performance plateaus at 15,000 data points.

9.2 Random Noise in Training Data

To identify how the Random Forest classifier reacts to the noise in training data, the model is evaluated with different percentages of random noise from 5% to 50% of mislabelled training data. For each percentage, the labels of embeddings generated from HinSAGE are randomly mislabelled and the Random Forest model is trained. Figure 9b shows that, the model is able to retain an F1 score above 0.9 for noise percentages below 3-4%, showing that the model is robust against random noise.



(a) (b)

Figure 9: (a) Accuracy of Random Forest classifier for different train set sizes of embeddings, (b) F1 score of Random Forest classifier for different levels of noise.

9.3 Adversarial Noise in Training Data

Studies [13] have shown deep learning based models can be fooled by adversarial examples. That is to embed carefully calculated perturbations such that they exploit the model's learned knowledge against itself. Furthermore, recent studies such as PR-BCD [30] attack expose that even the graph neural networks are susceptible to adversarial attacks. Hence, we carry out the following experiment to evaluate the possibility of an adversarial attack on our proposed graph neural network based model. First, we adversarially poison the DNS graph using the PR-BCD attack by utilizing the unsupervised learned model as the victim. PR-BCD attack is a scalable white-box poisoning attack that carries out edge additions and removals. We restrict perturbations only on the *Domain-resolves-IP* edges as PR-BCD is designed for homogeneous graphs. Figure 10, compares the performance of the original graph to the poisoned graph across varying perturbation rates (percentage to which is the number of *Domain-resolves-IP* either added or removed from the graph). Results show that the performance drop due to the adversarial attack is only marginally significant (0.015-0.025%). We attribute this observation to the unsupervised learning setting of the purposed model. Adversarial attacks such as PR-BCD target the model weights that are trained for a specific classification. Since our model doesn't require such optimization, the attack is unable to maximize its effectiveness. Furthermore, the neighborhood sampling in our GNN encoder reduces the effects of adversarial perturbations on other mechanisms such as GCN and RGCN. Thus, our overall model is relatively robust against well-known adversarial attacks such as PR-BCD.

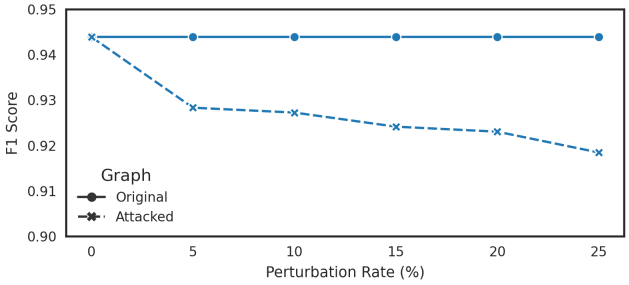


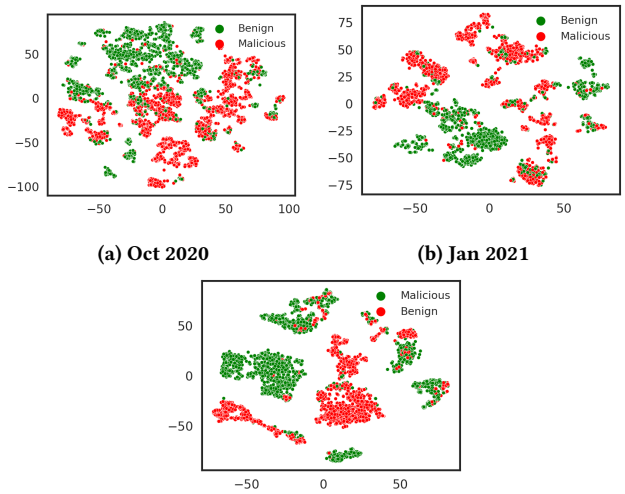
Figure 10: F1 score performance of the poisoned graph with increasing perturbation rates

9.4 Quality of the embeddings

Figure 11 shows the T-SNE plots of the embedding space of the benign and malicious domains for three different months in our dataset. The clear separation of two classes in these figures demonstrates that our approach can produce embeddings that can discriminate malicious from benign resulting in a classifier with high precision and a low FPR.

9.5 Sensitivity to parameters

Given that the proposed malicious domain detection model depends on two trainable models, it's vital to analyze their sensitivity to different parameters for both performance and reproducibility. Hence, the hyper-parameters of the models are defined by either empirical results or standards in the field. In Figure 12, we present the detection performance on 2020 Oct. data for three influential parameters while controlling the others. We observe that we get



(a) Oct 2020 (b) Jan 2021

(c) Apr 2021

Figure 11: T-SNE clustering of the generated domain embeddings (colored by the label)

the optimal results with 96 hidden parameters and 4 GNN layers. We use these tuned parameters for all the experiments.

10 LIMITATIONS AND FUTURE WORK

The proposed DNS graph doesn't utilize client information compared to other graph based detection systems. This is a result of utilizing only the publicly available information on a global scale. Positively, this method avoids any privacy violations that may occur if DNS logs are collected to capture client nodes and their relationships. One may augment the graph with such client information utilizing a privacy preserving technique to improve classification performance.

Our presented implementation utilized and is evaluated on a conservative set *Alexa top 1 million* domains as the benign ground truth. However, one could incorporate multiple benign domain lists, such as privately owned benign domain lists (e.g. Crunchbase company list), to cross-reference and further improve the ground truth quality. Additionally, the benign ground truth could be cross-checked against existing blocklists to eliminate potential malicious domains.

Our analysis shows that we do not have PDNS data for about 20% seed domains. While this does not affect the classification performance, this reduces the number of unseen malicious domains we predict. One may increase the coverage by considering additional passive DNS sources (e.g., VT PDNS, Rapid7 PDNS) or actively identified domains. Due to the short lifespan associated with many malicious domains, the latter approach should be performed frequently after a domain is registered.

Potential future work is to further analyze the behavior of IPs and ASNs. Exploring further upon the observation that adversaries reuse the infrastructure for multiple exploitations and high availability of malicious domains around specific IPs and ASNs, it is interesting to study the relationship between such infrastructure and hosting providers. We plan to cluster IPs and ASNs hosting malicious domains and analyze their interactions.

11 RELATED WORK

11.1 Graph Based Malicious Domain Detection

Nabeel et al. [24] utilizes the passive DNS data and graph inference algorithms such as belief propagation and path-based algorithms for malicious domain detection. MalRank [25] implements a large-scale graph inference algorithm to detect malicious domains in SIEM environments. The HIN includes node types such as domain, IP, ASN, and relationships in-between. HinDom [35] implements a malicious domain detection system by representing DNS in a Heterogeneous Information Network (HIN) and a meta-path based path-similarity classifier. Meanwhile, ours generate high-dimensional representations for the heterogenous components (i.e., domains, IPs, and ASNs) and fed into a trained domain classifier to classify the maliciousness of the domains. The HIN proposed by HinDom contains clients, domains, IP addresses, and their relationships collected from a local network. Thus, deploying on a global scale without privacy concerns is challenging. In contrast, our system seeds from existing malicious domains and expands the HIN using passive DNS at a larger scale. Furthermore, our system embeds ASN nodes in the knowledge graph, allowing us to include interactions between the IP nodes. Meanwhile, HGDom [37] utilizes a heterogeneous variant of GCN called MAGCN for domain classification. Ringer [23] proposes a scalable method to detect malicious domains by a dynamic graph neural network. DeepDom [36] uses SHetGCN to classify malicious domains, a heterogeneous variant of GCN. The graph contains clients, domains, IP addresses, accounts, and relationships in between. GAMD [43] and HANDDom [22] utilize variations of heterogeneous graph neural networks for malicious domain prediction. The DNS graph contains clients, domains, IPs, and relationships among them. Recently, Taeri et al. [18] proposed a graph based approach to detect phishing URLs. Even though these models show promising results, they have limitations when predicting at a global scale. Some approaches require capturing DNS server logs and DNS tracking. The collection of such data requires placing sensors at specific terminals. Thus, such approaches have serious concerns on privacy and scalability. The proposed model overcomes these issues by building the knowledge graph on publicly available information and improving the maliciousness of the graph. Further, most of these approaches work on static graphs and do not show to generalize to unseen data.

11.2 Predictive Block-listing

Felegyhazi et al. [11] propose an approach to predict additional malicious domains from seed malicious domains utilizing an unsupervised learner. Their observation is that domains registered within a short time period and resolving from the same authoritative name server are likely to be associated. In contrast, our approach relies on the hosting infrastructure and the features of domains/IPs in order to predict undiscovered malicious domains. Another line of work [34, 42] attempts to produce better blocklists by tailoring the global blocklists to different local settings. However, these approaches do not predict new malicious domains not present in reactive blocklists. EvilSeed [17] proposes an approach to crawl

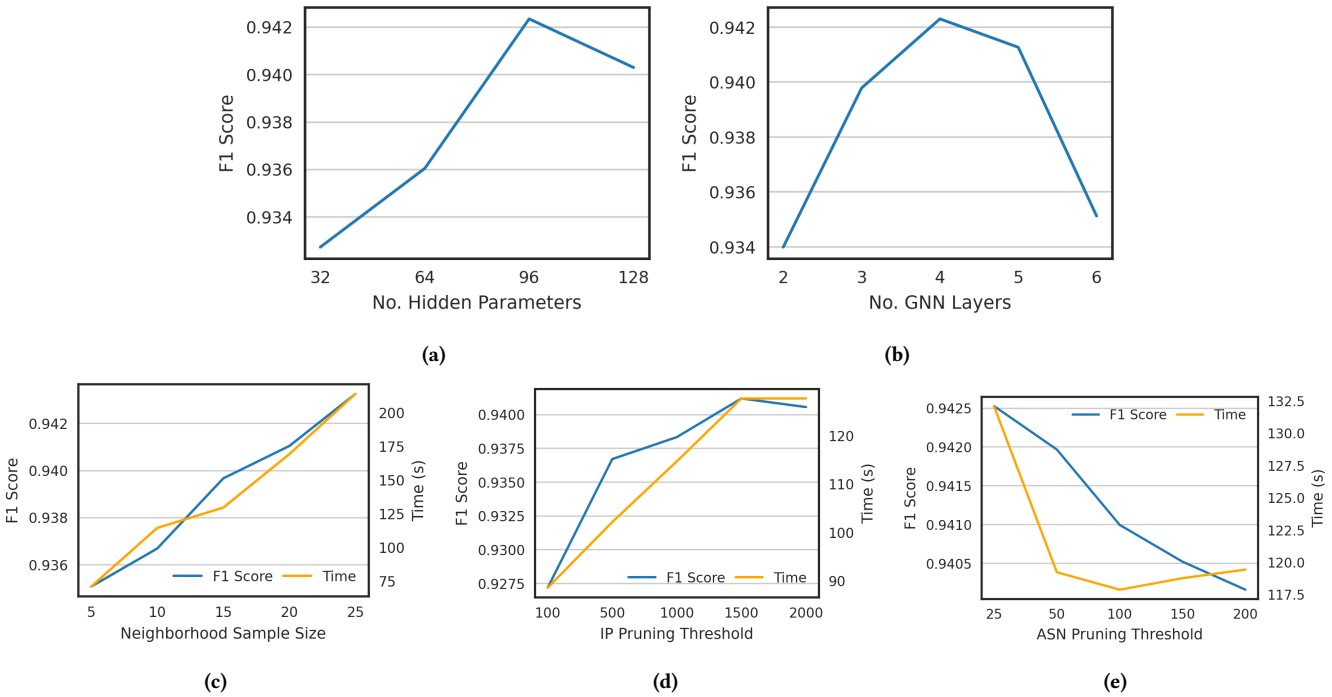


Figure 12: Results of parameter analysis.

for likely malicious websites given a set of small seeds of malicious websites. Their intuition is that malicious websites share similar templates and are likely to link together for easy management. In contrast, our approach does not rely on web content to link domains together and thus can detect malicious domains much earlier in their domain life cycle even if they are cloaked, reducing the window of vulnerability.

12 CONCLUSIONS

In this work, we propose a blocklist agnostic approach to turn traditionally reactive domain blocklists into predictive blocklists by identifying the hosting neighborhoods of malicious domains already seen in these blocklists. We first build a highly malicious DNS graph around the known malicious domains by performing a selective crawling of the PDNS repository. After careful post-processing, we learn deep graph embeddings of the domains and subsequently train a highly accurate supervised classifier over the embedding to proactively predict unseen malicious domains. Our experimental results show that our approach for VirusTotal feed detects on average 4.7 unseen malicious domains for every seed malicious domain at a very low FPR of 0.001. Further, we observe the concerning trend that 47% of predicted malicious domains that are later flagged in VirusTotal are identified only after more than 3 weeks to months since our model detects them. An interesting future direction is to augment the graph with additional information such as registration, and certificates, and further improve the performance.

ACKNOWLEDGMENTS

We would like to thank Qatar Computing Research Institute for providing access to the data used in this work and computing resources.

REFERENCES

[1] 2022. *Alexa Rank.* https://www.alexa.com/ Accessed: 12-10-2023.
[2] 2023. *OpenPhish.* https://openphish.com/ Accessed: 12-10-2023.
[3] 2023. *PhishTank.* https://phishtank.org/ Accessed: 12-10-2023.
[4] Bhupendra Acharya and Phani Vadrevu. 2021. PhishPrint: Evading Phishing Detection Crawlers by Prior Profiling. In *30th USENIX Security Symposium (USENIX Security 21).* USENIX Association, 3775–3792. https://www.usenix.org/conference/usenixsecurity21/presentation/acharya
[5] Leyla Bilge, Sevil Sen, Davide Balzarotti, Engin Kirda, and Christopher Kruegel. 2014. Exposure: A Passive DNS Analysis Service to Detect and Report Malicious Domains. *ACM Transactions on Information Systems Security* 16, 4, Article 14 (April 2014), 28 pages.
[6] Euijin Choo, Mohamed Nabeel, Doowon Kim, Ravindu De Silva, Ting Yu, and Issa Khalil. 2023. A Large Scale Study and Classification of VirusTotal Reports on Phishing and Malware URLs. *Proc. ACM Meas. Anal. Comput. Syst.* (2023). https://doi.org/10.1145/3626790
[7] Yuxiao Dong, Nitesh V. Chawla, and Ananthram Swami. 2017. Metapath2vec: Scalable Representation Learning for Heterogeneous Networks. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.* Association for Computing Machinery, New York, NY, USA, 135–144.
[8] Farsight Security, Inc. 2023. *DNS Database.* https://www.dnsdb.info/ Accessed: 10-01-2023.
[9] Edona Fasllija, Hasan Ferit Enişer, and Bernd Prünster. 2019. Phish-hook: Detecting phishing certificates using certificate transparency logs. In *International Conference on Security and Privacy in Communication Systems.* Springer, 320–334.
[10] Álvaro Feal, Pelayo Vallina, Julien Gamba, Sergio Pastrana, Antonio Nappa, Oliver Hohlfeld, Narseo Vallina-Rodriguez, and Juan Tapiador. 2021. Blocklist Babel: On the Transparency and Dynamics of Open Source Blocklisting. *IEEE Transactions on Network and Service Management* 18, 2 (2021), 1334–1349. https://doi.org/10.1109/TNSM.2021.3075552
[11] Mark Felegyhazi, Christian Kreibich, and Vern Paxson. 2010. On the Potential of Proactive Domain Blacklisting. In *Proceedings of the 3rd USENIX Conference on*

Large-Scale Exploits and Emergent Threats: Botnets, Spyware, Worms, and More (San Jose, California) *(LEET'10)*. USENIX Association, USA, 6.

[12] Matthias Fey and Jan E. Lenssen. 2019. Fast Graph Representation Learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*.

[13] I. Goodfellow, J. Shlens, and C. Szegedy. 2015. Explaining and Harnessing Adversarial Examples. In *ICLR*.

[14] Aditya Grover and Jure Leskovec. 2016. Node2vec: Scalable Feature Learning for Networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (San Francisco, California, USA) *(KDD '16)*. Association for Computing Machinery, New York, NY, USA, 855–864. https://doi.org/10.1145/2939672.2939754

[15] W. Hamilton, Z. Ying, and J. Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *NIPS*.

[16] Shuang Hao, Alex Kantchelian, Brad Miller, Vern Paxson, and Nick Feamster. 2016. PREDATOR: Proactive Recognition and Elimination of Domain Abuse at Time-Of-Registration. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. 1568–1579.

[17] L. Invernizzi, P. M. Comparetti, S. Benvenuti, C. Kruegel, M. Cova, and G. Vigna. 2012. EvilSeed: A Guided Approach to Finding Malicious Web Pages. In *2012 IEEE Symposium on Security and Privacy*. 428–442. https://doi.org/10.1109/SP.2012.33

[18] Taeri Kim, Noseong Park, Jiwon Hong, and Sang-Wook Kim. 2022. Phishing URL Detection: A Network-based Approach Robust to Evasion. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security* (Los Angeles, CA, USA) *(CCS '22)*. Association for Computing Machinery, 1769–1782.

[19] T. Kipf and M. Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *ICLR*.

[20] Neeraj Kumar, Sukhada Ghewari, Harshal Tupsamudre, Manish Shukla, and Sachin Lodha. 2021. When Diversity Meets Hostility: A Study of Domain Squatting Abuse in Online Banking. In *2021 APWG Symposium on Electronic Crime Research (eCrime)*. 1–15. https://doi.org/10.1109/eCrime54498.2021.9738769

[21] Udesh Kumarasinghe, Fatih Deniz, and Mohamed Nabeel. 2022. PDNS-Net: A Large Heterogeneous Graph Benchmark Dataset of Network Resolutions for Graph Learning. *arXiv preprint arXiv:2203.07969* (2022).

[22] Zhiping Li, Fangfang Yuan, Yanbing Liu, Cong Cao, Fang Fang, and Jianlong Tan. 2022. Heterogeneous Graph Attention Network for Malicious Domain Detection. In *Artificial Neural Networks and Machine Learning - ICANN 2022 - 31st International Conference on Artificial Neural Networks, Bristol, UK, September 6-9, 2022, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 13530)*, Elias Pimenidis, Plamen P. Angelov, Chrisina Jayne, Antonios Papaleonidas, and Mehmet Aydin (Eds.). Springer, 506–518. https://doi.org/10.1007/978-3-031-15931-2_42

[23] Zhicheng Liu, Shuhao Li, Yongzheng Zhang, Xiaochun Yun, and Chengwei Peng. 2020. Ringer: Systematic Mining of Malicious Domains by Dynamic Graph Convolutional Network. In *Computational Science – ICCS 2020*, Valeria V. Krzhizhanovskaya, Gábor Závodszky, Michael H. Lees, Jack J. Dongarra, Peter M. A. Sloot, Sérgio Brissos, and João Teixeira (Eds.). Springer International Publishing, Cham, 379–398.

[24] Mohamed Nabeel, Issa M. Khalil, Bei Guan, and Ting Yu. 2020. Following Passive DNS Traces to Detect Stealthy Malicious Domains Via Graph Inference. *ACM Trans. Priv. Secur.* 23, 4 (2020).

[25] Pejman Najafi, Alexander Mühle, Wenzel Pünter, Feng Cheng, and Christoph Meinel. 2019. MalRank: A Measure of Maliciousness in SIEM-Based Knowledge Graphs. In *Proceedings of the 35th Annual Computer Security Applications Conference* (San Juan, Puerto Rico, USA) *(ACSAC '19)*. Association for Computing Machinery, New York, NY, USA, 417–429. https://doi.org/10.1145/3359789.3359791

[26] Adam Oest, Penghui Zhang, Brad Wardman, Eric Nunes, Jakub Burgis, Ali Zand, Kurt Thomas, Adam Doupé, and Gail-Joon Ahn. 2020. Sunrise to Sunset: Analyzing the End-to-end Life Cycle and Effectiveness of Phishing Attacks at Scale. In *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, 361–377. https://www.usenix.org/conference/usenixsecurity20/presentation/oest-sunrise

[27] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.

[28] Quirin Scheitle, Oliver Hohlfeld, Julien Gamba, Jonas Jelten, Torsten Zimmermann, Stephen D. Strowes, and Narseo Vallina-Rodriguez. 2018. A Long Way to the Top: Significance, Structure, and Stability of Internet Top Lists. In *IMC*.

[29] M. Schlichtkrull, T. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling. 2018. Modeling Relational Data with Graph Convolutional Networks. In *The Semantic Web*. 593–607.

[30] T. Schmidt, H. Şirin, D. Zügner, A. Bojchevski, and S. Günnemann. 2021. Robustness of Graph Neural Networks at Scale. In *NeurIPS*.

[31] M. Sharif, J. Urakawa, N. Christin, A. Kubota, and A. Yamada. 2018. Predicting Impending Exposure to Malicious Content from User Behavior. In *CCS*. 1487–1501.

[32] Chuan Shi, Yitong Li, Jiawei Zhang, Yizhou Sun, and Philip S. Yu. 2017. A survey of heterogeneous information network analysis. *IEEE Transactions on Knowledge and Data Engineering* 29, 1 (2017), 17–37. https://doi.org/10.1109/TKDE.2016.2598561

[33] Ravindu De Silva, Mohamed Nabeel, Charith Elvitigala, Issa Khalil, Ting Yu, and Chamath Keppitiyagama. 2021. Compromised or Attacker-Owned: A Large Scale Classification and Study of Hosting Domains of Malicious URLs. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 3721–3738. https://www.usenix.org/conference/usenixsecurity21/presentation/desilva

[34] Fabio Soldo, Anh Le, and Athina Markopoulou. 2010. Predictive Blacklisting as an Implicit Recommendation System. In *2010 Proceedings IEEE INFOCOM*. 1–9. https://doi.org/10.1109/INFCOM.2010.5461982

[35] Xiaoqing Sun, Mingkai Tong, Jiahai Yang, Liu Xinran, and Liu Heng. 2019. HinDom: A Robust Malicious Domain Detection System based on Heterogeneous Information Network with Transductive Classification. In *22nd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2019)*. USENIX Association, Chaoyang District, Beijing, 399–412. https://www.usenix.org/conference/raid2019/presentation/sun

[36] Xiaoqing Sun, Zhiliang Wang, Jiahai Yang, and Xinran Liu. 2020. Deepdom: Malicious domain detection with scalable and heterogeneous graph convolutional networks. *Computers and Security* 99 (2020), 102057. https://doi.org/10.1016/j.cose.2020.102057

[37] Xiaoqing Sun, Jiahai Yang, Zhiliang Wang, and Heng Liu. 2020. HGDom: Heterogeneous Graph Convolutional Networks for Malicious Domain Detection. In *IEEE/IFIP NOMS*. 1–9. https://doi.org/10.1109/NOMS47738.2020.9110462

[38] Petar Veličković, William Fedus, William L. Hamilton, Pietro Liò, Yoshua Bengio, and R Devon Hjelm. 2019. Deep Graph Infomax. In *International Conference on Learning Representations*. https://openreview.net/forum?id=rklz9iAcKQ

[39] L. Wang, A. Nappa, J. Caballero, T. Ristenpart, and A. Akella. 2014. WhoWas: A Platform for Measuring Web Deployments on IaaS Clouds. In *Proceedings of the 2014 IMC*. 101–114.

[40] Florian Weimer. 2005. Passive DNS Replication. In *FIRST Conference on Computer Security Incident*. 98.

[41] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. 2021. A Comprehensive Survey on Graph Neural Networks. *IEEE Trans. Neural Networks Learn. Syst.* 32, 1 (2021), 4–24. https://doi.org/10.1109/TNNLS.2020.2978386

[42] Jian Zhang, Phillip Porras, and Johannes Ullrich. 2008. Highly Predictive Blacklisting. In *Proceedings of the 17th Conference on Security Symposium* (San Jose, CA) *(SS'08)*. USENIX Association, USA, 107–122.

[43] Shuai Zhang, Zhou Zhou, Da Li, Youbing Zhong, Qingyun Liu, Wei Yang, and Shu Li. 2021. Attributed Heterogeneous Graph Neural Network for Malicious Domain Detection. In *2021 IEEE 24th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*. 397–403.

[44] Yury Zhauniarovich, Issa Khalil, Ting Yu, and Marc Dacier. 2018. A Survey on Malicious Domains Detection through DNS Data Analysis. *ACM Comput. Surv.* 51, 4, Article 67 (jul 2018), 36 pages. https://doi.org/10.1145/3191329